

Digital Saarthi: Technical Architecture & System Guide

End-to-End System Design, AI Model Integration, Implementation Details & Reliability Safeguards

1. Executive Overview

Digital Saarthi serves as an AI-powered *Action-Guidance Layer* tailored specifically for underserved, elderly, and low-literacy citizens seeking government welfare services. Rather than attempting to duplicate or replace large government aggregator portals (such as UMANG or DigiLocker), Digital Saarthi acts as a digital bridge. It transforms complex bureaucratic notifications, confusing portals, and eligibility documents into step-by-step, verified actions using accessible voice, OCR scanning, and factual rule-backed retrieval.

2. System Architecture & Component Graph

The architecture is modularly separated into an Accessibility-First Frontend, an API/Orchestration Gateway, specialized AI processing engines, and a deterministic safety core.

Figure 1: High-level data and control flow across frontend, backend inference modules, rule engine, and LLM explanation layer.

3. Detailed Component Breakdown & AI Models

• Accessibility-First Frontend (React / Flutter)

Provides high-contrast, large touch targets, single-tap voice capture, and camera/upload integration for documents. It abstracts digital complexity by guiding users visually and audibly through each stage of interaction.

• Speech-to-Text: OpenAI Whisper / Bhashini

How it works: Utilizes an encoder-decoder Transformer trained on extensive multilingual speech audio. Converts speech waveforms to log-mel spectrograms, which are processed into linguistic tokens.

Why it is helpful: Enables voice interaction in local vernacular languages and accents, eliminating literacy barriers and handling background noise effectively.

• Document OCR: Tesseract / EasyOCR

How it works: Performs image binarization, adaptive thresholding, line/word segmentation, and neural character recognition (LSTM/CRNN) to extract raw text from captured Aadhaar cards, BPL cards, and ration cards.

Why it is helpful: Removes manual data entry friction for elderly citizens by parsing fields like Age, DOB, Gender, and BPL status via structured regex extraction.

• Intent Detection & Request Router

How it works: Employs lightweight sentence embedding matching / classification to route citizen queries (e.g., pension inquiry vs ration status vs document verification) to the correct processing pipeline.

Why it is helpful: Ensures fast routing, low latency, and deterministic workflow selection without relying on expensive end-to-end model calls.

- **Deterministic Rule Engine (The Safety Core)**

How it works: Implements hardcoded, auditable boolean evaluation logic based directly on gazetted eligibility criteria (e.g., IGNOAPS: Age >= 60 AND BPL == True).

Why it is helpful: Eliminates hallucinations in eligibility determination. The model never guesses eligibility; the Rule Engine authoritatively computes it.

- **Retrieval-Augmented Generation (RAG Knowledge Base)**

How it works: Indexes verified official government scheme documentation into a vector database (e.g., ChromaDB/FAISS) with dense retrieval embeddings.

Why it is helpful: Grounding LLM responses strictly in retrieved official context ensures high factual fidelity, up-to-date scheme requirements, and source reference traceability.

- **LLM Explanation Layer (e.g., Llama 3 / Claude / GPT)**

How it works: Acts strictly as a natural language synthesizer and action translator. It receives the deterministic Rule Engine outcome and retrieved RAG context to format a conversational, compassionate, step-by-step guide.

Why it is helpful: Formats complex eligibility logic into clear, non-jargon steps in the user's native dialect while preventing ungrounded advice.

4. Backend API Endpoints Reference

The backend is built with FastAPI for asynchronous performance, OpenAPI schema validation, and low-latency response times.

Endpoint	Method & Payload	Function & Response
/api/voice-query	POST { "query": "..."} 	Detects intent from speech text, queries RAG, returns guidance text and suggested UI actions (e.g. scan document).
/api/scan-document	POST (Multipart) file=@Aadhaar.jpg	Extracts textual fields via OCR, parses age, DOB, BPL card status, and returns structured user attributes.
/api/check-eligibility	POST { "scheme": "ignoaps", "age": 72, "has_bpl": true }	Executes deterministic rule verification. Returns eligibility boolean, detailed rationale, and concrete application steps.
/api/schemes	GET ?query=pension	Lists matching government schemes with eligibility criteria, required documents, and benefit descriptions.

5. Step-by-Step Implementation & Run Guide

Step 1: Backend Setup & Execution

Navigate to the backend directory, install dependencies, and launch FastAPI with Uvicorn:

```
cd backend
pip install -r requirements.txt
python -m app.main # Running on http://localhost:8000
```

Step 2: Frontend Setup & Execution

Navigate to the frontend directory, install Node dependencies, and start the development server:

```
cd frontend
npm install
npm start # App running on http://localhost:3000
```

Step 3: Verification with cURL

Test backend endpoints directly:

```
curl -X POST http://localhost:8000/api/check-eligibility ^
-H "Content-Type: application/json" ^
-d "{\"scheme\":\"ignoaps\", \"age\":72, \"has_bpl\":true}"
```

6. Anti-Hallucination & Privacy Safeguards

To ensure high trust and avoid common pitfalls of AI in civic tech, Digital Saarthi enforces three foundational pillars:

- 1. **Zero Decision Delegation:** LLMs never make eligibility judgments. Judgments are made purely by deterministic code.
- 2. **PII Protection:** User documents are processed ephemerally in-memory and never sent to external public LLM training queues.
- 3. **Source Grounding:** Every guidance recommendation links back to verifiable official government department guidelines.